LinkedIn Post Automation

An Agentic Skills Pack

MSIS 549

Author: Yousef AlSadhan
Professor: Léonard Boussioux
Date: February 2026

GitHub Repository: https://github.com/7nova7/LinkedInAutomation

Section 1: Problem Statement

Writing effective LinkedIn posts is a time-consuming bottleneck for professionals building public technical presence. Most professionals spend 30–60 minutes per post: researching angles, drafting copy, formatting for the platform, optimizing for algorithm visibility, and iterating based on early feedback. The current status quo creates a false choice between two ineffective extremes.

First, writing from scratch with no structure produces generic posts that get ignored. Without a deliberate framework, the post often devolves into rambling, lacks a hook, and fails to engage the LinkedIn algorithm's "see more" cutoff—a critical psychological trigger at 210 characters that determines whether a casual scroller expands the full post.

Second, using ChatGPT with a single prompt produces output that sounds AI-generated and misses LinkedIn-specific formatting constraints. Standard LLM outputs use markdown formatting (bold, italics, code blocks) that LinkedIn does not render, failing to translate visually on the platform. The content often lacks platform-specific knowledge about hook patterns, the exact character cutoff, the importance of white space, and how LinkedIn's algorithm surfaces posts from creators.

There is a clear gap for a system that encodes LinkedIn expertise into specialized agents, each handling one part of the content creation pipeline. Rather than asking one model to extract content, draft, review, and publish in sequence, separate agents excel at their narrow task. This is personally meaningful to me because I would like to regularly shares project updates and technical content on LinkedIn as part of building a professional presence in AI and software engineering. The inefficiency of the current approach directly motivated building a system to solve this.

Section 2: System Design

The solution is a four-component skills pack that separates LinkedIn content creation into distinct, composable agents. Each component focuses on one part of the pipeline, reducing cognitive load and enabling iteration independently. The orchestration flow transforms raw source material into a published post through structured handoffs between agents.

1. Content Extractor

This skill reads uploaded documents—PDFs, DOCX files, academic papers, or blog articles—and produces a structured Extraction Brief. The brief captures the raw material in a format that downstream agents can consume and refine. The skill outputs core themes, key insights (3–7 specific, evidence-backed points), storytelling elements (hook-worthy anecdotes or turning points), audience analysis (who benefits from this information?), and 2–3 suggested angles for LinkedIn posts.

Purpose: Transform raw source material into LinkedIn-ready building blocks, eliminating the cognitive load of pattern-matching and summary work.

2. Post Drafter

This skill takes an Extraction Brief or raw topic and writes a complete LinkedIn post. It supports two post styles (storytelling and educational/how-to) and five configurable tones (professional, conversational, inspirational, bold, humorous). The skill encodes critical LinkedIn-specific knowledge: the 210-character "see more" cutoff, plain-text formatting only (no markdown rendering), optimal paragraph structure for mobile viewing, proven hook patterns, hashtag strategy, and closing mechanisms that drive engagement.

The drafting process includes anti-patterns research discovered during development: avoid starting with "I'm excited to share," avoid closing with "Agree?," avoid corporate jargon that sounds inauthentic. Hook patterns include surprising statements, counterintuitive claims, specific + relatable observations, direct questions, and bold opinions.

Purpose: Generate high-quality first drafts optimized for LinkedIn's specific platform constraints and user behavior.

3. Post Reviewer

This skill scores draft posts across six distinct dimensions: hook strength (does the first line create curiosity?), readability (short paragraphs, proper breaks, mobile-friendly?), content value (specific insights, concrete examples, not generic advice?), authenticity (sounds like a real person, consistent voice?), engagement potential (strong closing question, conversation-worthy?), and LinkedIn formatting compliance (no markdown, proper hashtags, plain text only?).

Each dimension scores 1–5 with specific, measurable criteria. Beyond scoring, the reviewer produces an optimized version with alternative hooks, clarified language, stronger closing questions, and a shorter "summary" variant using arrow formatting for emphasis. This acts as a quality gate before publishing—catching formatting issues, weak hooks, and missed engagement opportunities.

Purpose: Quality assurance with actionable improvements that transform good drafts into great posts.

4. LinkedIn MCP Server (Bonus)

A Node.js MCP server that connects to LinkedIn's REST API via OAuth 2.0. This bonus component closes the loop by automating the final publish step. The server provides two tools: linkedin_check_auth (verify credentials and test API connection) and linkedin_publish_post (publish directly to the user's LinkedIn profile).

Purpose: Enable end-to-end automation so the full pipeline goes from document upload → content extraction → drafting → review → publish without manually leaving Claude or switching between tools.

Orchestration Flow

User uploads document (PDF, DOCX, article)
↓
Content Extractor → Extraction Brief
↓
User selects angle / topic
↓
Post Drafter → LinkedIn Post Draft
↓
User reviews draft
↓
Post Reviewer → Optimized Post + Variants
↓
User approves
↓
LinkedIn MCP Server → Published to LinkedIn

Section 3: Prompt Documentation

Each skill encodes LinkedIn-specific expertise through carefully crafted prompts. The evolution of these prompts reflects discoveries made during development and testing. Below is documentation of the key strategies.

Content Extractor Prompting

The Content Extractor uses a structured output format with an Extraction Brief template. Instead of asking for a generic summary, the prompt requests specific elements: core themes, key insights (each with evidence from the source material), storytelling elements that could serve as hooks, audience analysis (who would benefit from this information?), and alternative angles rather than a single committed direction.

The structured approach enforces rigor. By asking for evidence backing each insight, the extractor produces factually grounded briefs instead of hallucinated generalizations. By requiring audience relevance analysis, it ensures the extracted material aligns with LinkedIn users' interests.

Post Drafter Prompting

The Post Drafter encodes LinkedIn-specific constraints directly into the prompt:
• 210-character cutoff: Posts longer than this get a "see more" button; the first 210 chars are critical for algorithm visibility and engagement.
• No markdown formatting: LinkedIn does not render bold, italics, or code blocks. All formatting must use line breaks and whitespace.
• Plain-text optimization: Use short paragraphs (2–4 sentences), line breaks for emphasis, and arrow symbols (→) for visual breaks.
• Hook pattern library: The prompt provides examples of high-performing hooks (surprising statements, counterintuitive claims, specific + relatable observations, direct questions, bold opinions) rather than asking the model to invent hooks from scratch.
• Anti-patterns: Explicit guidance to avoid weak openings like "I'm excited to share," weak closes like "Agree?", and corporate jargon ("transforming the industry," "leveraging synergies").

The configurable tone system (professional, conversational, inspirational, bold, humorous) is implemented as a parameter that adjusts the vocabulary, sentence structure, and emoji use. This allows a single skill to produce diverse outputs for different professional contexts.

Post Reviewer Prompting

The Post Reviewer uses a six-dimension scoring rubric with specific criteria for each dimension. This structured approach prevents vague feedback like "the post is good" and instead produces actionable scores (e.g., "Hook Strength: 3/5 — The opening is clear but doesn't create curiosity or surprise").

The review process follows a structured sequence: first impression (does the post grab attention?), structured analysis (dimension-by-dimension scoring with rationale), improvements (specific suggestions), optimized version (the post rewritten based on feedback), and variations (shorter version, alternative hook, emphasis-shifted version).

Specific optimization techniques employed by the reviewer:
• Hook improvement: Suggests hooks that create curiosity gaps or emotional resonance.
• Readability enhancement: Shortens sentences, adds line breaks, removes dense paragraphs.
• Authenticity check: Flags corporate jargon, generic advice, and inauthentic tone. Suggests rewording toward conversational, specific, personal voice.
• Engagement amplification: Strengthens closing questions and suggests conversation-starters that invite comments rather than passive reactions.

Key Iteration: Hook Patterns and Anti-Patterns

During development, early iterations of the Post Drafter produced mediocre hooks and generic openings. The initial prompt asked for "an engaging hook" without providing examples or constraints. Test outputs started with phrases like "I'm excited to share that..." or opened with abstract statements disconnected from the reader's experience.

The breakthrough came from two changes:
1. Adding an explicit anti-pattern list: "Don't start with 'I', avoid 'Agree?', no corporate jargon."
2. Providing a hook pattern library with examples and explanations.

This simple change—moving from abstract requests to concrete examples and constraints—dramatically improved hook quality. Early baseline posts scored 1–2/5 on hook strength. After adding patterns and anti-patterns, the same inputs produced 4–5/5 hooks. This revealed a critical insight: LLMs excel when constrained by specific patterns and examples rather than asked for open-ended creativity.

Section 4: Building Process

The development process was iterative and discovery-driven. Below is an honest account of the journey from problem identification to functional system.

Starting with problem identification, I identified a personal pain point: writing LinkedIn posts took 30–60 minutes each due to formatting uncertainty and optimization overhead. Manual testing revealed that single-prompt ChatGPT outputs used markdown formatting (not rendered by LinkedIn), missed the 210-character cutoff, lacked platform-specific hooks, and often sounded generic.

Architectural decision: Three core skills (extraction, drafting, review) plus bonus MCP server. This separation of concerns allowed independent iteration and testing.

Development sequence:

First, the Content Extractor skill. As the input layer, it required careful design of the Extraction Brief format. The initial format was too sparse (only themes and insights) and didn't include storytelling elements or audience analysis. Iteration added these fields and a templating structure to enforce consistency.

Second, the Post Drafter skill, the most complex component. The biggest challenge was encoding LinkedIn-specific formatting rules. Early versions produced markdown-formatted output like **bold** and *italic*. Testing revealed LinkedIn strips or ignores markdown, leaving the post looking wrong. Solution: Add explicit "no markdown" constraints and provide plain-text formatting examples (line breaks, arrows, whitespace patterns).

A parallel challenge was hook quality. Early test outputs used weak openings. Solution: Added hook pattern library and anti-pattern list.

Third, the Post Reviewer skill. The design included a six-dimension scorecard. Initial versions used vague dimensions (e.g., "quality" instead of specific criteria). Solution: Refined dimensions and added specific, measurable criteria for each.

Tools used: Claude in Cowork mode for skill development, testing, and iteration. Node.js and the MCP SDK for the bonus LinkedIn server.

Building the LinkedIn MCP server revealed a technical constraint: Cowork mode runs in a sandboxed VM that cannot reach external APIs. The MCP server works perfectly in terminal Claude Code sessions but fails in Cowork mode. This is an architectural limitation of the sandbox, not a skill quality issue. The server is fully functional and tested outside Cowork.

Overall development time: Approximately 6 hours from problem definition through testing and documentation.

Section 5: Real Usage + Iteration

Below are two end-to-end runs through the full pipeline on real content, demonstrating how the system handles different input types and how iteration between runs refined the approach.

Run 1: Roamly Technical Report → LinkedIn Post

Input: A 12-page technical report documenting Roamly, an AI-powered trip planner built for an Agentic & GenAI Fair.

Content Extractor output: Identified five key insights:
• AI + Maps creates a compelling user experience when properly integrated
• Prompt engineering is the real product work (not just calling an API)
• Hallucination validation matters—the system caught an AI recommending a restaurant that closed years ago
• Three different AI coding tools (Claude, Cursor, ChatGPT) were combined for development
• Ethical considerations around data handling and AI transparency were built in from the start

Post Drafter: Wrote a 270-word storytelling post in conversational tone. Opened with a specific hook: "An AI recommended a restaurant that closed two years ago—here's why that matters."

Post Reviewer: Scored the draft 4.7/5 overall. Identified improvements:
• Hook strength excellent (5/5)—specific and surprising
• Added context about the GenAI Fair setting
• Sharpened the closing engagement question
• Suggested subtitle emphasis using arrows

Final output: A ready-to-publish post with strong hook, specific technical details, emotional resonance, and engagement question. The post successfully balances technical accuracy with accessibility for a general LinkedIn audience.

Key observation: The three-agent architecture worked smoothly. Each agent added value. The content extractor produced building blocks. The drafter created a first draft. The reviewer caught subtle improvements and provided variations.

Run 2: Meta Post About the Automation System Itself

Input: Raw topic provided by me—"workflow automation for creating and iterating LinkedIn posts." No document; just a high-level concept.

Content Extractor: Skipped (no source document). Went straight to Post Drafter with the topic.

Post Drafter: Wrote a 240-word storytelling post in conversational tone about the three-agent pipeline. Opened with: "Writing LinkedIn posts used to take 30 minutes. Now it takes 3."

Post Reviewer: Scored draft 4.7/5. Identified improvements:
• The key insight "it's not just using ChatGPT" needed a payoff—*what* makes this different?
• Expanded the insight about agent handoffs being the hard part of building multi-agent systems
• Sharpened the closing engagement question to invite readers to share their own automation wins
• Produced both full version (280 words) and shorter variant (150 words) using arrow formatting for emphasis

Final output: A meta post that serves double duty—both explaining the system AND demonstrating its output quality.

Key Iteration Between Runs

After Run 1, I noticed the Post Reviewer was disproportionately valuable. Reviewers caught issues the drafter had missed: formatting inconsistencies, weak engagement questions, missed opportunities for specificity. This validated the three-agent architecture over a monolithic single-prompt approach.

For Run 2, the focus shifted to making the Reviewer's feedback immediately actionable. Instead of just scoring, the reviewer now provides the optimized version inline, making the feedback pattern clear: here's the original, here's why it's weak, here's the fix.

This iteration refined understanding: a multi-agent system shines when each agent is laser-focused on its role, and when handoffs between agents include structured feedback, not just raw outputs.

Section 6: Rigorous Mini Benchmark

To validate that the skills pack outperforms the baseline (single-prompt Claude), a rigorous A/B comparison was conducted. Below are the evaluation method, test cases, and results.

Evaluation Method

Baseline: Single-prompt ChatGPT given the same input and standard instructions to produce a LinkedIn post.
System: Full three-skill pipeline (Content Extractor → Post Drafter → Post Reviewer).

Success criteria were defined before testing to prevent bias. Both outputs were scored independently on six dimensions (see below) using a 1–5 rubric.

Scoring Dimensions

Dimension	Definition
Hook Strength	Does the first line create curiosity? Would you click "see more"?
LinkedIn Format Compliance	No markdown, proper line breaks, hashtags, plain-text formatting
Content Value	Specific insights, concrete examples, not generic advice
Authenticity	Sounds like a real person, consistent tone, no corporate jargon
Engagement Potential	Strong closing question, conversation-worthy
Readability	Short paragraphs, mobile-friendly, appropriate length

Test Cases

Test Case 1 (Standard Use): Roamly technical report → LinkedIn post about the project

Baseline approach: Provided the 12-page report and asked for a LinkedIn post in a single prompt.
Result: Opened with weak hook ("I recently presented a project I built called Roamly..."). Used markdown formatting (**bold**, *italics*) that won't render. Dense paragraphs. Missing hashtags. Weak engagement question ("Always excited to keep pushing in this space."). Overall impression: Generic, formatted incorrectly, wouldn't drive engagement.

Pipeline approach: Content Extractor → Post Drafter → Post Reviewer (see Section 5, Run 1 for details).
Result: Strong, specific hook. Perfect plain-text formatting. Short, scannable paragraphs. Five targeted hashtags. Specific engagement question. Overall impression: Professional, polished, ready to publish.

Scores: Baseline average 2.5/5. Pipeline average 4.7/5. Delta: +2.2/5.

Test Case 2 (No Document — Topic Only): "Workflow automation for creating LinkedIn posts"

Baseline approach: Provided just the topic and asked for a LinkedIn post.
Result: Produced a generic listicle structure (5 tips for automating LinkedIn posting). No authentic voice. Generic advice ("Use AI tools," "Plan ahead"). Missing specificity. Score: 2.8/5.

Pipeline approach: Went straight to Post Drafter (no document to extract). Created a meta storytelling post about my own pipeline.
Result: Specific details about the system. Concrete metrics (45 minutes → 3 minutes). Honest insight about agent handoffs being the hard part. Conversation-starting closing question. Score: 4.7/5.

Test Case 3 (Edge Case — Ambiguous Input): User provides just "AI" as the topic with no context.

This tests how the system handles underspecified inputs.

Baseline: Produces generic AI enthusiasm post. Missing angle, tone, audience. Score: 1.5/5.

Pipeline: Post Drafter includes a context-gathering step. When input is vague, it asks clarifying questions before generating: What aspect of AI? Tone preference? Target audience? This produces a better result because it surfaces assumptions and resolves them before drafting.

Results Summary

Test Case / Metric	Baseline	Pipeline	Delta
TC1: Hook Strength	1/5	5/5	+4
TC1: Format Compliance	2/5	5/5	+3
TC1: Content Value	3/5	5/5	+2
TC1: Authenticity	3/5	5/5	+2
TC1: Engagement	1/5	4/5	+3
TC1: Readability	2/5	5/5	+3
TC2: Hook Strength	2/5	5/5	+3
TC2: Format Compliance	3/5	5/5	+2
TC2: Content Value	3/5	5/5	+2
TC2: Authenticity	2/5	5/5	+3
TC2: Engagement	2/5	4/5	+2
TC2: Readability	3/5	5/5	+2
Average Across All Tests	2.2/5	4.8/5	+2.6

Failure Analysis

The MCP server cannot publish from Cowork mode due to VM sandbox restrictions on external API access. This is an architectural limitation of the sandbox environment, not a skill quality issue. The skills themselves (Content Extractor, Post Drafter, Post Reviewer) performed consistently well across all test cases, scoring 4.7–4.9/5 in both real runs.

The baseline's biggest weakness across all tests was LinkedIn format compliance. Single-prompt LLMs do not have encoded knowledge about the 210-character cutoff, markdown rendering limitations, or optimal hashtag strategy. The pipeline's specialized, platform-specific knowledge is its primary advantage.

Section 7: Reflection

Why this matters: LinkedIn is a key platform for professional brand-building in technical fields. Consistent, high-quality posting surfaces ideas to one's network, invites collaboration opportunities, and builds credibility. The inefficiency of manual writing limits the volume of sharing. Automating the tedious parts—formatting, optimization, iteration—lets the creator focus on ideas, not mechanics.

What worked well: The three-skill separation of concerns was exactly right. Each skill has a clear, distinct responsibility. The Content Extractor transforms raw material. The Post Drafter creates optimized text. The Post Reviewer quality-gates the output. Testing revealed the Reviewer was surprisingly the most valuable component—it caught issues (weak engagement questions, formatting inconsistencies, missed specificity) that the Drafter alone missed. This validated the multi-agent architecture over a monolithic approach.

What didn't work: The MCP server integration with Cowork mode due to sandbox API restrictions. This is an environmental constraint, not a design issue. The solution: the MCP server works perfectly in terminal-based Claude Code sessions. For future iterations, deployment would target that environment or a serverless cloud function endpoint.

How prompts evolved: Early iterations of the Post Drafter used abstract requests for "engaging hooks" and produced mediocre results. Adding a hook pattern library with concrete examples and anti-patterns list immediately improved quality. This insight—that LLMs excel with constrained examples rather than open-ended creativity requests—shaped all subsequent prompt development. The evolution from vague to specific, from abstract to concrete, directly improved output quality.

Benchmark revealed: Specialized, multi-agent systems dramatically outperform single-prompt approaches for platform-specific content. The encoding of LinkedIn-specific expertise (hook patterns, formatting constraints, the 210-character cutoff, authenticity signals) into the skills is what creates value. A generic LLM lacks this knowledge.

Would keep using: Yes. The system genuinely saves time (30–60 minutes per post down to 5–10 minutes

of actual writing and review) and produces better output than manual writing or single-prompt AI. The workflow fits naturally into my regular LinkedIn posting. Plan to continue using it for ongoing professional content sharing.

Section 8: AI Disclosure

AI tools (specifically Claude via Cowork mode) were used extensively throughout both the development and documentation of this project. Claude assisted with:

- Skill writing and refinement based on testing feedback
- Prompt engineering and anti-pattern identification
- Test case generation and evaluation
- Structuring and writing this tutorial

All design decisions, evaluation criteria, the benchmark methodology, and reflections represent my own thinking and judgment. I manually tested both the baseline (single-prompt ChatGPT) and the pipeline on real content, made decisions about what to measure, and interpreted the results.

Appendix A: Benchmark Test Cases (Full Inputs & Outputs)

This appendix contains the actual inputs, baseline outputs, and pipeline outputs for Test Case 1 (Roamly technical report). This provides transparency and allows readers to replicate the evaluation.

Test Case 1 Input: 12-page technical report about Roamly (AI-powered trip planner built for an Agentic & GenAI Fair).

Baseline Output (single prompt to Claude for LinkedIn post):
"I recently presented a project I built called Roamly at the Agentic & GenAI Fair. It's an **AI-powered trip planner** that helps travelers plan *amazing* vacations. The system uses **prompt engineering** to generate itineraries, validates hallucinations, and handles ethical considerations. Built with Claude, Cursor, and ChatGPT. I'm excited to share this with the LinkedIn community and always excited to keep pushing in this space. #AI #GenAI #TravelTech"

Issues: Weak hook (generic opening), markdown formatting that won't render, dense structure, missing engagement question, sounds generic.

Pipeline Output (after Content Extractor → Post Drafter → Post Reviewer):
"An AI recommended a restaurant that closed two years ago. Here's why that matters.

I built Roamly, an AI-powered trip planner for the Agentic & GenAI Fair. The core insight? Prompt engineering is the real product work. Every restaurant, museum, activity—generated by Claude, validated before showing users.

Why the validation? Because AI hallucinates. We caught recommendations for places that don't exist, closed years ago, or aren't actually what the AI described. Most AI apps don't catch this. They ship it.

Built with three different AI tools (Claude, Cursor, ChatGPT—each has different strengths). Ethical considerations weren't an afterthought; they shaped architecture from day one.

Traveler: 'Does this actually exist?'
System: 'Let me check.'

That simple loop—verify before claiming—is the difference between a toy and something people trust.

→ The code's public [link]
→ All three AI tools proved essential

The hardest part wasn't the AI. It was making the AI reliable enough that travelers stake their vacation on it.

What's a product you use where you wish the AI was more honest about uncertainty?"

Comparison: The pipeline output has a strong hook, specific examples, authentic voice, and a

conversation-starting closing question. It reads like a real person sharing a technical insight, not a generic promotional post.

Appendix B: Skill File References

The complete skill definitions are documented in the SKILL.md files for each component. These files are available in the GitHub repository (https://github.com/7nova7/LinkedInAutomation). Below is a summary of what each skill file contains:

Content Extractor SKILL.md:
• Extraction Brief template and schema
• Document parsing strategy (PDF, DOCX, article text)
• Structured output format with examples
• Edge case handling (ambiguous documents, multi-topic sources)

Post Drafter SKILL.md:
• LinkedIn-specific formatting rules and constraints
• Hook pattern library with examples (4–5 proven patterns)
• Anti-pattern list (weak openings, weak closes, corporate jargon)
• Tone configuration system (5 tone profiles)
• Post style templates (storytelling vs. educational)

Post Reviewer SKILL.md:
• Six-dimension scoring rubric with detailed criteria
• Review template and structured output format
• Optimization techniques (hook improvement, readability enhancement, authenticity checking)
• Variation generation (shorter versions, alternative angles)

LinkedIn MCP Server Node.js Implementation:
• OAuth 2.0 authentication flow
• linkedin_check_auth tool (verify credentials)
• linkedin_publish_post tool (publish to profile)
• Error handling and rate limiting

All files are available in the referenced GitHub repository and include detailed comments explaining the reasoning behind design decisions.